

Exercise 7

Manipulating data types

A binary tree

In lectures we saw (repeatedly) a binary tree with values at nodes and empty leaves. This time, we're going to make a tree that holds values at nodes and leaves, and then do some manipulation of it.

Our data type is

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
  deriving (Eq, Show)
```

The template `trees-exercise7.hs` also defines some helper functions to construct trees of various kinds

```
-- Produce a "balanced" tree with the specified number of nodes
makeBalancedTree :: Int -> Tree Int
-- Take a tree and produce a new tree that is mirror-symmetric
mirrorTree :: Tree a -> Tree a
```

We'll first implement some type class instances for our data type.

Exercise 7.1

Implement `Functor` and `Foldable` instances for the `Tree` data type.

Show that your implementation of `fmap` obeys the two functor laws.

Hint: We'll use these instances to implement functionality transforming trees later, so if you couldn't figure it out, don't forget that you can add

```
{-# LANGUAGE DeriveFunctor #-}
{-# LANGUAGE DeriveFoldable #-}
```

At the top of your file and add `deriving (Functor, Foldable)` to the `data` declaration.

To help construct trees, we'll build minimal depth trees from lists with `toTree`

Exercise 7.2

Define

```
toTree :: [a] -> Tree a
```

That takes a list and builds a tree of minimal depth.

You may find `splitAt` useful.

One thing you should check is that `toList . toTree` is the identity on lists.

To check that you got things right, also define

```
depth :: Tree -> Int
```

that returns the depth of a tree, defined as the length of its longest branch from the root to any leaf.

You should then find that the depth of a tree constructed with `toTree` grows like `logBase 2` of the length of the input list.

We'll call binary trees *balanced* if the number of leaves in the left and right subtrees of every `Node` differs by at most one (with leaves being trivially balanced).

Exercise 7.3

Define a function

```
balanced :: Tree a -> Bool
```

that determines if a tree is balanced or not.

Hint: Since your datatype is `Foldable` you can compute its `length`.

Question 7.4

What complexity does your implementation of `balanced` have? Does it traverse the tree only once (so linear complexity in the tree size), or does it traverse many times?

Hint: If you used `length`, think about how that is implemented.

If you traversed more than once, can you think of a way to define `balanced` that only visits each node in the tree at most once?

Hint: Think about what information you need to return at each level in the recursion so that you don't need to go back down later to reconstruct things.

We'll call binary trees *symmetric* if we can draw a vertical line through the root node such that right subtree is the mirror image of the left subtree.

Exercise 7.5

Write a function

```
symmetric :: Tree a -> Bool
```

that checks if a binary tree is structurally symmetric (that is the shape is symmetric, but the values may differ).

Hint: You might find it helpful to write

```
mirror :: Tree a -> Tree a -> Bool
```

to check if two trees are the mirror image of one another.

A simple calculator

We will now build a tiny domain-specific language for integer arithmetic expressions, and an evaluator for this language. We begin by only supporting addition:

```
data Expr = Val Int | Add Expr Expr
  deriving (Eq, Show)
```

This says that an expression is either a `Val` (which contains an `Int`) or else an `Add` which contains two expressions. We make the expression representing $(1+(4+5))$ like so:

```
expr = (Add (Val 1) (Add (Val 4) (Val 5)))
```

First, we will write some individual recursive functions to inspect these expressions in some way. Then we will look at ways of generalising the idea using higher-order functions.

Exercise 7.6

Define

1. Evaluation of an expression

```
eval :: Expr -> Int
```

2. Collecting a list of all `Val` nodes

```
collect :: Expr -> [Val]
```

3. Counting how many `Val` nodes there are

```
size :: Expr -> Int
```

In writing these functions, you should see that you produce the same pattern each time. The `Val` node is replaced by some new value constructed from the integer inside it, and the `Add` node is replaced by some new value constructed from the replacement of both its internal nodes. We will encapsulate this abstraction in the function

```
folde :: (Int -> a) -> (a -> a -> a) -> Expr -> a
```

That is, `folde` eats a function `Int -> a` that converts `Val` nodes into type `a`, a function `a -> a -> a` that converts `Add` nodes into type `a` by eating both their converted children, and an `Expr`, returning a value of type `a`.

As an example, we can implement `eval` using this new function like so:

```
eval :: Expr -> Int
eval expr = folde id (+) expr
```

This says: “replace `Val` nodes with their contents, and replace `Add` nodes with the summation of their contents”.

Exercise 7.7

Implement `folde` and then the functions `collect` and `size` using it.